

Aftershock

A maintenance workflow for the bug that survives the fix. The incident ticket is closed. Sibling functions still fail the same way.

Bob reads the incident and closes that class. A local scan finds the siblings and spends 0 Bobcoins.

PROBLEM

The patch is local. The fault is not.

A carrier retry was fixed in delivery confirmation. Pickup, customs release, and exception handling still apply the same event twice.

The same split shows up in naive clocks and in two writes with no rollback. The next page is a sibling, not a new bug.

Name every sibling. Close the class.

Aftershock reads the incident note, classifies the shape, and lists every function that still matches. Closed anchors stay closed.

The board is the handoff. Each open site becomes one Bob subagent task, with the file, the line, and the change.

Bob closes. The scan only points.

The Aftershock skill reads the incident. Agent mode runs the scanner, then three subagents write one replacement each, in parallel.

That split keeps the 40 Bobcoins on reading and patching. Search does not spend them. The rule file blocks the same shape on the next edit.

PROOF ON THE SAMPLE

3 anchors held. 8 siblings did not.

Harborline is a fictional parcel hub. The incident notes are original and contain no personal data.

Tests show delivery retry is ignored, pickup retry is not, the closed clock is timezone-aware, and a failed manifest rolls back only on the closed path.

WHO IT IS FOR

Teams who close incidents.

Application and platform engineers use it after a production fix, before the same shape ships again in another module.

Run **python3 -m aftershock serve** and open the board. One incident in Bob is enough to show the parallel close.